SAHAYAK

MASTER DEVELOPER GUIDE

MR. JOHAN JYOTI KULI

LEAD DEVELOPER | COMPLETE PROJECT WALKTHROUGH

Study guide and speaker script - from zero to the full system

WHAT YOU WILL MASTER

- Project purpose and honest scope
- Computer vision, landmarks, features, and machine learning
- Static input: 126 values | Sequence input: 1,320 values
- Frontend, API, backend, models, database, and deployment
- Testing, privacy, troubleshooting, and difficult judge questions

THE COMPLETE FLOW

1. **CAMERA**
2. **HAND LANDMARKS**
3. **NUMERIC FEATURES**
4. **BACKEND API**
5. **TRAINED MODEL**
6. **PREDICTED LABEL**
7. **WEBSITE DISPLAY**

Reference for the lead presenter - use measured results only.

Repository: github.com/Mrjohanjyotikuli/SAHAYAK-ISL

SAHAYAK MASTER DEVELOPER AND PRESENTATION GUIDE

MR. JOHAN JYOTI KULI | LEAD DEVELOPER
Study guide and speaker script - from zero to the full system

1. WHAT THIS GUIDE IS FOR

This is your personal master guide as lead developer and presenter. The other three guides give each teammate a speaking role; this guide goes deeper so you can explain the complete system, answer technical questions, and understand the limits of every claim.
Your responsibility is to understand each component, why it exists, what information it receives, what it returns, and what still needs work. You do not need to memorise every line of code, but you should understand the whole data flow.
Be accurate about your individual contribution. Credit external libraries and teammates. Do not claim you invented MediaPipe, machine learning, or the general field of sign-language recognition.

2. YOUR OPENING PRESENTATION SCRIPT

Good morning respected judges, teachers, and everyone present here. My name is Johan Jyoti Kuli, and I am the lead developer of SAHAYAK, a real-time Indian Sign Language recognition prototype. The project integrates a web interface, camera-based hand landmark detection, machine-learning prediction, and supporting website functions.
Our goal is to explore one focused accessibility challenge: whether a camera and a trained model can recognise a limited set of hand signs and present the predicted label in a usable interface. I will explain how the user's hand becomes numerical features, how the model predicts, how the website and backend communicate, how the system should be tested, and what needs to improve.
Adapt statements about your individual contribution so they reflect the work you personally completed.

3. THE PROBLEM - AND SAHAYAK'S SCOPE

People who use sign language may face communication barriers when others do not understand their language. Sign languages are full languages with their own grammar and variation. Recognising a few isolated hand signs is not the same as understanding a full signed conversation.
SAHAYAK is a prototype for a limited set of trained labels. It is not a universal translator and should not be described as understanding all ISL sentences, facial grammar, regional variation, or every two-handed movement. It is not a replacement for interpreters or a guaranteed communication service.
Optional context: The 2011 Census of India recorded 5,071,007 people in its "disability in hearing" category. This is a historical census category, not the number of ISL users and not a current population estimate. Census source:
https://censusindia.gov.in/nada/index.php/catalog/43369/
WHO hearing-loss context:
https://www.who.int/news-room/fact-sheets/detail/deafness-and-hearing-loss

4. STARTING FROM ZERO - KEY TERMS

Computer vision means using computers to interpret images or video. In this project, the camera provides video frames and a hand-landmark detector estimates key points on visible hands.
A landmark is a reference point, such as a fingertip or joint. MediaPipe Hands detects up to 21 landmarks for each hand. A feature is a measurable value that a model processes; x, y, and z landmark coordinates are features in this pipeline.
Machine learning means fitting a model to examples so it can learn statistical patterns and make predictions on new inputs. Training fits the model using examples. Inference means using the trained model to make a prediction for an input it has not necessarily seen before.
A dataset is a collection of labelled examples. A model artifact stores a trained model and related parameters. A label list names possible classes, but editing a label list does not teach the model a new sign.
The frontend is what the user sees in the browser. The backend is the server-side application. An API route is a defined address through which the frontend and backend exchange data.
A confidence score is model-specific and is not a guarantee of correctness. A threshold may reject weak predictions, but it does not remove errors or replace formal evaluation.

5. COMPLETE ARCHITECTURE - FOLLOW ONE PREDICTION

Step 1 - Camera: the browser asks for camera permission and receives a video stream. Frame quality depends on lighting, distance, camera angle, and whether the hands are visible.
Step 2 - Landmark detection: MediaPipe Hands runs in the browser and estimates hand key points. The website draws the landmark overlay locally; the overlay is not the classification result.
Step 3 - Feature preparation: JavaScript and backend utilities must agree on the number, order, normalisation, and meaning of feature values. If formats differ, a model may receive incorrect inputs even when the overlay looks correct.
Step 4 - Request: the browser sends a feature vector to a backend API route. The deployed backend uses Python and Flask. It loads a trained model artifact and runs inference.
Step 5 - Prediction: the model returns a predicted label and any available confidence information. Application logic may apply a threshold or reject an uncertain result.
Step 6 - Display: the frontend receives the response and updates the result shown to the user. Supporting speech or translation functions may also be available, depending on the feature and network access.
Memorise the flow: CAMERA -> MEDIAPIPE LANDMARKS -> NUMERIC FEATURES -> BACKEND API -> TRAINED MODEL -> PREDICTED LABEL -> WEBSITE DISPLAY.

6. STATIC RECOGNITION - WHY 126 VALUES?

For the two-hand static pathway, each hand has 21 landmarks. Each landmark has three coordinates: x, y, and z. One hand therefore has 21 x 3 = 63 values. Two hands have 2 x 21 x 3 = 126 values.
The 126 values are the length of a feature vector. They are not 126 signs, 126 training samples, or an accuracy score.
The current intended label metadata includes HELLO, I_LOVE_YOU, NO, THANK_YOU, and YES. The live backend status and actual model classes are authoritative for the deployed system. A label appearing in metadata alone does not prove that the class is recognised reliably.
Correct landmarks do not guarantee correct classification. Errors may arise from unfamiliar hand shapes, orientation, occlusion, missing hands, mismatched feature preparation, or visually similar classes.

7. SEQUENCE RECOGNITION - WHY TIME MATTERS

A single frame captures a moment. Some signs depend on movement over time, so the project also has a sequence pathway. The sequence feature utility uses 132 values per frame: the 126 landmark coordinates plus six wrist-position values.
The sequence is resampled into 10 time bins. The resulting input is 132 x 10 = 1,320 values.
This is the designed input format; it does not prove every continuous sign has been learned correctly. Performance depends on labelled sequences, order, segment boundaries, preprocessing, training quality, and testing on unseen examples.
For a judge: "Static recognition classifies features from a frame. Sequence recognition uses a time-ordered representation of multiple frames."

8. TECHNOLOGY STACK - WHAT EACH PIECE DOES

Frontend: HTML defines page structure, CSS handles layout and visual styling, and JavaScript handles interactions, camera workflow, landmark drawing, feature preparation, and API requests. The frontend is hosted on Vercel.
Hand detector: MediaPipe Hands estimates landmarks in the browser. Detection and drawing are separate from backend classification.
Backend: Python and Flask provide API routes, validate or prepare inputs, load models, run predictions, and support other functions. The backend is hosted on Render.
Model artifacts: the project contains backend/model/sahayak_twohand.joblib and backend/model/sahayak_sequence.joblib. The current artifacts use scikit-learn/joblib. Do not call the system TensorFlow-based or a CNN unless the implementation is changed and verified.
Storage: SQLite is used for continuous gesture samples. Database rows and model training data are not necessarily the same thing; verify how a model was trained before making a dataset claim.
Configuration: frontend/config.js points the browser at the backend API URL. Render configuration and requirements control backend deployment. A service worker may cache static page assets, but caching the page does not make remote inference work offline.

9. IMPORTANT API ROUTES TO KNOW

/api/health - checks whether the service responds. It does not prove the model is accurate.
/api/status - provides current service/model and project status information. Prefer live status for current labels or counts instead of inventing figures.
/api/predict-features - accepts prepared static feature input from the current workflow and returns a prediction response.
/api/predict - a legacy or alternate prediction endpoint retained by the backend. Trace the frontend to know which route is used for an actual request.
/api/sequence/predict - handles a sequence feature representation.
/api/sequence/samples - supports sequence-sample access and recording as implemented by the backend. Public sample submission is not model training by itself.
/api/sequence/export - exports sequence data using the supported backend behaviour.
/api/translate and /api/speech - support translation and speech-related features. Online services may need internet access.
Know both the route name and the real request structure. Inspect frontend/studio.html for fields sent by the browser and backend/web_app.py for validation, response shape, model selection, and error handling.

10. DATA, TRAINING, TESTING AND ACCURACY

To add a new sign, define its label and meaning; collect multiple correctly labelled examples from consenting signers; include useful variation; inspect ambiguous samples; and preserve the feature format expected by the model.
Train a candidate model in a controlled environment, then test it on a separate holdout set.
Prefer separating people or recording sessions between training and testing. If near-duplicate frames from one recording appear in both sets, results may be inflated by data leakage.
Report a per-class breakdown, confusion matrix, and the number and source of test examples. Record lighting, camera, signer diversity, and whether the test was static or sequence-based. Measure inference latency separately from classification quality.
Do not claim a percentage such as 98 percent unless documented evaluation supports it. API route timing measures server-side execution, not necessarily total camera-to-result latency and not model accuracy.
Adding a class to labels_twohand.json is not enough. A compatible model must be trained or updated, tested, packaged, deployed, and rechecked. The sequence model needs correctly prepared ordered sequence data.

11. PRIVACY, CONSENT AND RESPONSIBLE USE

The browser should request camera permission. Explain whether data is sent to a server, whether samples are stored, and how a person can avoid dataset collection.
Collect training data with informed permission. Store only what is needed, protect access, and avoid keeping identifiable images or videos unnecessarily. Landmark coordinates can still be sensitive in context; numerical data is not automatically anonymous.
Because the model can fail, users should confirm uncertain outputs. Never position SAHAYAK as a substitute for qualified interpreters, especially in medical, legal, emergency, or other high-stakes communication.

12. DEPLOYMENT - FROM CODE TO A LIVE WEBSITE

The frontend and backend are deployed separately. Vercel hosts the frontend; Render hosts the Python Flask backend. The frontend configuration file supplies the backend API base URL.
For a frontend change, review HTML, CSS, JavaScript, asset paths, and the service-worker cache version where relevant. Commit to the branch that triggers deployment, check Vercel build status, then test the live page in a fresh browser session.
For a backend or model change, verify dependencies, model paths, API behaviour, and the Render start configuration. Deploy, call health and status routes, then test a real prediction from Live Studio.
Use the browser developer tools network panel to check request URL, HTTP status, request body, response body, and JavaScript errors. A page that loads successfully does not prove that the backend or model works.

13. TROUBLESHOOTING - WHAT TO CHECK FIRST

Camera unavailable: check browser permission, operating-system privacy settings, and whether another app is using the camera. Camera access normally requires HTTPS outside localhost.
Landmarks absent or misaligned: check that MediaPipe assets loaded, the correct video element is used, the overlay canvas matches display dimensions, and coordinates are mapped correctly.
Landmarks look right but prediction fails: check exact feature order, normalisation, input length, selected model, label mapping, response shape, and confidence handling. Detector success and classifier success are separate stages.
Network or API error: check frontend/config.js, the backend URL, Render service availability, CORS, browser network panel, and backend logs. A slow cold start on a hosted service is not a measure of accuracy.
A new sign is never predicted: verify training examples exist, the trained model includes the class, feature shape matches the model, the new artifact is deployed, and /api/status agrees with expectations.
Sequence prediction is unstable: check frame collection, sequence length, segment boundaries, time order, resampling, wrist values, and consistency with training preprocessing.
Backup demo: keep a screenshot or a consented recording and the architecture diagram ready. Clearly tell judges when you show a recording rather than a live result.

14. HARD JUDGE QUESTIONS - LEAD DEVELOPER

Q: What is your main contribution? A: Describe the parts you personally integrated or built, such as frontend workflow, landmark detection integration, API connection, deployment, or supporting functions. Credit external tools and teammates.
Q: Is SAHAYAK a complete ISL translator? A: No. It predicts among a limited set of trained labels. Full sign-language understanding involves language structure, facial expression, context, and movement beyond this scope.
Q: Why MediaPipe? A: It offers hand landmarks as a compact representation that can be converted into model features.
Q: Why 126 values? A: Two hands x 21 landmarks per hand x three coordinates per landmark equals 126.
Q: Why 1,320 sequence values? A: 132 values per frame multiplied by 10 time bins equals 1,320.
Q: How do you know it is accurate? A: Report only a documented evaluation on separate test examples, with the test size and per-class results. A live demo is insufficient.
Q: What is innovative? A: The project integrates camera-based landmark detection, model inference, a deployed website, and supporting functions into an educational prototype. Do not claim this project invented MediaPipe or the broader technology.
Q: Why separate frontend and backend? A: The browser handles camera interaction and visual feedback; the server loads the model and returns predictions.
Q: Does it work offline? A: Not completely. The browser may draw landmarks locally after its assets load, but backend predictions and some translation or speech functions need connectivity.
Q: What if it predicts the wrong sign? A: Record the failure and investigate conditions and model behaviour rather than hiding the error.
Q: How add a new sign? A: Collect consented labelled examples, preserve feature format, train a compatible model, evaluate unseen samples, compare with existing classes, deploy, and retest.
Q: How protect privacy? A: Request permission, explain processing and storage, collect samples with consent, minimise retention, and restrict dataset access.
Q: Can it be trusted in emergencies? A: Not as a guaranteed service. The vocabulary is limited and errors are possible; it must not replace qualified communication support.

15. YOUR FULL PRESENTATION CLOSING

As lead developer, I want to emphasise that SAHAYAK is not just a label on a screen. It is an end-to-end prototype and a way to learn how much careful work accessible technology needs before it can be dependable.
Our next priorities are to improve data diversity, evaluate every label fairly, strengthen sequence recognition, report performance transparently, and gather feedback from people who use Indian Sign Language. We will treat limitations and errors as evidence for improvement, not something to hide.
Thank you for your time. I am happy to answer questions about architecture, model inputs, testing, deployment, limitations, and future improvements.

16. FINAL LEAD-DEVELOPER CHECKLIST

Practise the opening without reading. Know the 126 and 1,320 calculations. Test the deployed website and backend. Test at least two supported labels. Verify camera permission. Confirm the displayed label agrees with the backend response. Prepare screenshots for failure scenarios.
Have one honest answer ready for metrics: "We have not completed a sufficiently documented evaluation for that number yet." That answer is stronger than inventing a result.
Have a clear ownership statement ready: describe work you personally completed, mention third-party tools, and credit your teammates.

PROJECT REFERENCE MAP

Repository: https://github.com/Mrjohanjyotikuli/SAHAYAK-ISL
Frontend home: SAHAYAK-Vercel-Deployment/frontend/index.html
Live Studio: SAHAYAK-Vercel-Deployment/frontend/studio.html
Frontend API configuration: SAHAYAK-Vercel-Deployment/frontend/config.js
Backend/API routes: SAHAYAK-Vercel-Deployment/backend/web_app.py
Static features: SAHAYAK-Vercel-Deployment/backend/features.py
Sequence features: SAHAYAK-Vercel-Deployment/backend/sequence_features.py
Sequence sample database utilities: SAHAYAK-Vercel-Deployment/backend/sequence_db.py
Model artifacts: SAHAYAK-Vercel-Deployment/backend/model/
Label metadata: SAHAYAK-Vercel-Deployment/backend/labels_twohand.json
Deployment notes: SAHAYAK-Vercel-Deployment/README-DEPLOY.md and DEPLOYMENT-CHECKLIST.md